Functional Programming Practical Week 8

Configured Functions and Native Integration

50 tasks and five worked examples. Complete the class set below; the remaining tasks are additional practice. Project integration tasks can be adapted to your final project.

Class work

Suggested class set: Tasks 01–06, with the stated checks and one additional boundary or failure assertion for each task. The instructor may replace these with tasks suited to the group. Intermediate tasks are further practice. Choose one of Tasks 46–49 for project extension; completing all 50 tasks is not required.

Native tasks require the supplied library and a C/C++ compiler. Complete setup before class or use these as an optional extension. Do not execute broken bindings or unbounded recursion before repairing them.

Use the Checks project as an assertion-runner example. Its Task 01 checks intentionally fail against the broken starter. Repair Task 01, rerun the checks, then add tests for the other selected tasks.

DIFFICULTY	TASKS	COUNT
BASIC	01–20	20
INTERMEDIATE	21–40	20
ADVANCED	41–50	10

TASK FORMAT	COUNT	PURPOSE
FIX CODE	15	Identify the bug and repair observable behaviour
COMPLETE TODO	15	Complete the supplied contract
IMPLEMENT	15	Design a small operation from a stated contract
INTEGRATE	5	Apply the techniques in one project module

Material

Overview

Pass policies as functions and build stable configured functions through partial application and currying. Test composition order and avoid capturing mutable configuration. Keep native loading and binding in a separate adapter; native interoperability is not itself a functional technique.

Higher-order functions

A function is higher-order when it accepts or returns a function. Apply(items,score) must actually invoke score. Merely declaring a Func variable without using it does not change the application. Keep domain transformations pure; use counters only in test probes.

Partial application

Given Price(baseFee,rate,distance), fixing baseFee and rate produces distance -> price. This is partial application even if the original function is not curried. The resulting delegate must preserve the supplied stable values.

Currying

Currying rewrites a multi-argument function as successive single-argument functions. Func<int,Func<int,int>> means first receive one int, then return a function receiving another int. A curried three-argument price can be called as Price(baseFee)(rate)(distance). A factory returning a function can be useful without a general-purpose curry framework.

Composition

If the first function returns the type required by the second, compose them into a new function. Document execution order. Bind remains appropriate when a stage returns Result; ordinary composition is for ordinary successful values.

C and C# boundary

The supplied native library exports weighted_score with two int32_t arguments and an int32_t return. Inputs are 0..100 points and 1..10 weight; invalid input returns -1. The C# binding uses matching int parameters and Cdecl. Build for your OS and architecture. Loading is an effect; a deterministic numerical native function may still be used through a boundary adapter. Native interoperability itself is not a functional technique.

Working conventions

Use .NET 8 or later and SharedModels.cs. Native sources and GCC or Clang build commands are in Native. Compile the native library before Example05 and native tasks; provide its absolute path to Example05. Tasks 01 to 20 are basic, 21 to 40

intermediate, and 41 to 50 advanced. All fix and TODO snippets are members of TaskNN classes unless a nested type is shown.

Shared teaching types

These are reference and starter types, not a fully validated production domain. Tasks may ask you to strengthen or replace them. Some starter fragments need a different signature after repair. Each TaskNN class is isolated; remove its wrapper when moving operations into your project. Implement-type tasks provide the contract without a solution.

```
public record Delivery(string Id, int Km);
public record Product(string Id, int Quantity, decimal Price);
public record Candidate(string Id, int Points);
public record Pricing(decimal BaseFee, decimal Rate);
public abstract record Result<T>
{
    private protected Result() { }
    public sealed record Ok(T Value) : Result<T>;
    public sealed record Fail(string Error) : Result<T>;
}
public static class ProvidedNative
{
    [System.Runtime.InteropServices.DllImport("practice",
        EntryPoint="weighted_score",
        CallingConvention=System.Runtime.InteropServices.CallingConvention.Cdecl)]
    public static extern int Score(int points,int weight);
}
```

Worked examples

Example 01 A function supplies the rule

Use the same higher-order transformation with two simple policies.

```
int[] input = { 2, 3 };
Console.WriteLine(string.Join(",", Apply(input, x => x * 2)));
Console.WriteLine(string.Join(",", Apply(input, x => x + 10)));

public static int[] Apply(int[] values, Func<int,int> rule)
    => values.Select(rule).ToArray();
```

Expected result

```
4,6
12,13
```

Explanation

1. Apply receives data and behaviour explicitly. 2. The traversal is shared while each rule changes the result. 3. ToArray creates a result snapshot. 4. Neither policy changes input or depends on mutable settings.

Example 02 Partially apply a price

Fix base fee and rate to create a distance-only pricing function.

```
var standard = MakePrice(500m, 100m);
var express = MakePrice(800m, 150m);
Console.WriteLine(standard(2));
Console.WriteLine(express(2));

public static decimal Price(decimal basis, decimal rate, int km)
    => basis + rate * km;
public static Func<int,decimal> MakePrice(decimal basis, decimal rate)
    => km => Price(basis, rate, km);
```

Expected result

```
700
1100
```

Explanation

1. Price has three explicit arguments. 2. MakePrice fixes the first two. 3. The returned function receives only distance. 4. Two stable functions can coexist without editing a global rate.

Example 03 Read a nested Func type

Curry addition, then apply its first argument to obtain an increment-by-five function.

```
Func<int,Func<int,int>> add = a => b => a + b;
Func<int,int> addFive = add(5);
Console.WriteLine(addFive(3));
Console.WriteLine(add(2)(4));
```

Expected result

```
8
6
```

Explanation

1. The first call receives a and returns a function. 2. The second call receives b and produces the value. 3. addFive remembers a=5. 4. The captured int is stable; the calculation has no side effects.

Example 04 Compose a configured score and a label

Apply a configured score first and then turn it into a display label.

```
Func<int,int> score = points => points * 3;
Func<int,string> label = value => $"Score:{value}";
var pipeline = Compose(score, label);
Console.WriteLine(pipeline(4));

public static Func<T,V> Compose<T,U,V>(Func<T,U> first, Func<U,V> second)
    => value => second(first(value));
```

Expected result

```
Score:12
```

Explanation

1. The first stage returns int, which the second accepts. 2. Compose returns a new function without processing input yet. 3. Calling pipeline runs stages in first-then-second order. 4. Each stage can be tested independently.

Example 05 A real native function contributes a value

Build Native/practice.c and pass the absolute library path as the first command-line argument. Load that file for the practice import and call weighted_score.

```
if (args.Length != 1)
    throw new ArgumentException("Pass the absolute native library path");
string libraryPath = Path.GetFullPath(args[0]);
System.Runtime.InteropServices.NativeLibrary.SetDllImportResolver(
    typeof(ProvidedNative).Assembly, (name, assembly, searchPath) =>
        name == "practice"
            ? System.Runtime.InteropServices.NativeLibrary.Load(libraryPath)
            : IntPtr.Zero);
Console.WriteLine(ProvidedNative.Score(10, 3));
Console.WriteLine(ProvidedNative.Score(-1, 3));
```

Expected result

```
30
-1
```

Explanation

1. The resolver makes the loaded path explicit for this example process. 2. The C and C# signatures agree on 32-bit integers. 3. Valid input produces the deterministic numerical result. 4. The native rejection sentinel must be translated by a production adapter.

Practice tasks

Task 01 Invoke the supplied policy

Basic | Fix code | Higher order

Fix Apply so it uses the supplied rule instead of a hard-coded doubling policy.

```
public static int[] Apply(int[] xs,Func<int,int> rule)
    => xs.Select(x=>x*2).ToArray();
```

Required checks Input[2,3],+1 -> [3,4]; same input,*3 -> [6,9]; input unchanged.

Task 02 Return a configured multiplier

Basic | Complete TODO | Partial application

Complete MultiplyBy(int factor) returning a stable Func<int, int>.

```
public static Func<int,int> MultiplyBy(int factor)
{
    // TODO return a function accepting the remaining value.
    throw new NotImplementedException();
}
```

Required checks MultiplyBy(3)(4) -> 12; factor 0, input 9 -> 0; two factors coexist.

Task 03 Curry a subtraction

Basic | Implement | Currying

Implement Func<int, Func<int, int>> for a-b. Show separate calls for the first and second arguments.

Required checks Subtract(10)(3) -> 7; Subtract(3)(10) -> -7.

Task 04 Keep the fixed parameter stable

Basic | Fix code | Partial application

Fix Make so changing a static factor later does not change previously returned functions.

```
public static int Factor;
public static Func<int,int> Make(int factor)
{
    Factor=factor; return x=>x*Factor;
}
```

Required checks Make(3), change global factor 9, call 4 -> 12.

Task 05 Curry three price arguments

Basic | Complete TODO | Currying

Complete Price as baseFee -> rate -> km -> total, using decimal for fees and non-negative km.

```
public static Func<decimal,Func<decimal,Func<int,decimal>>> Price()
{
    // TODO return successive functions and validate distance.
    throw new NotImplementedException();
}
```

Required checks Price(500)(100)(2) -> 700; km 0 -> 500; negative km rejected explicitly.

Task 06 A supplied predicate selects candidates

Basic | Implement | Higher order

Implement Select(Candidate[], Func<Candidate, bool>) preserving input order. Return a new array.

Required checks Points[20,40,60],>=40 -> [40,60]; predicate false -> []; input unchanged.

Task 07 Fix composition order

Basic | Fix code | Composition

Fix Compose to run first then second; use functions where order changes the result.

```
public static Func<int,int> Compose(Func<int,int> first,Func<int,int> second)
    => x=>first(second(x));
```

Required checks First +1 then *2 on 3 -> 8; reversed would be 7 and must not occur.

Task 08 Partially apply a threshold

Basic | Complete TODO | Partial application

Complete AtLeast(int minimum) returning Func<Candidate, bool>.

```
public static Func<Candidate,bool> AtLeast(int minimum)
{
    // TODO capture minimum and return a predicate.
    throw new NotImplementedException();
}
```

Required checks Minimum 40 accepts 40 and 60, rejects 39; changing another threshold has no effect.

Task 09 A curried equality comparer

Basic | Implement | Currying

Implement a curried ordinal string equality function expected -> actual -> bool. Define null equality as both null only.

Required checks "A"/"A" -> true; "A"/"a" -> false; null/null -> true.

Task 10 Match the native export name

Basic | Fix code | Native

Fix the binding to the supplied weighted_score export. Preserve 32-bit int parameters and return type, and the Cdecl calling convention.

```
[System.Runtime.InteropServices.DllImport("practice",
    EntryPoint="weight_score",
    CallingConvention=System.Runtime.InteropServices.CallingConvention.Cdecl)]
static extern int Score(int points,int weight);
```

Required checks Native Score(10,3) -> 30; Score(0,1) -> 0; invalid input -> -1.

Task 11 Translate a native rejection

Basic | Complete TODO | Native

Complete ConvertNative(int value): non-negative value becomes Ok; -1 becomes Fail("native input"). Other unexpected negatives become a distinct protocol failure.

```
public static Result<int> ConvertNative(int value)
{
    // TODO distinguish success, rejection and unexpected negative.
    throw new NotImplementedException();
}
```

Required checks 30 -> Ok 30; 0 -> Ok 0; -1 -> input failure; -2 -> protocol failure.

Task 12 Build and check the native library

Basic | Implement | Native

Compile supplied practice.c for your platform. Call weighted_score from C# and document compiler command, architecture, binary path, and one manually checked result.

Required checks 10,3 -> 30; 100,10 -> 1000; 101,1 -> -1.

Task 13 Do not change the original array

Basic | Fix code | Higher order

Fix Transform to return a new array after applying f without overwriting xs.

```
public static int[] Transform(int[] xs,Func<int,int> f)
{
    for(int i=0;i<xs.Length;i++) xs[i]=f(xs[i]);
    return xs;
}
```

Required checks [1,2],+3 -> [4,5], original remains[1,2]; [] -> [].

Task 14 Return a configured formatting function

Basic | Complete TODO | Partial application

Complete Label(prefix) returning value -> string. Preserve the supplied prefix exactly.

```
public static Func<int,string> Label(string prefix)
{
    // TODO return the remaining-argument function.
    throw new NotImplementedException();
}
```

Required checks Label("Score:")(12) -> Score:12; empty prefix -> 12.

Task 15 Curried interval membership

Basic | Implement | Currying

Implement minimum -> maximum -> value -> bool for an inclusive interval. Reject maximum<minimum when applying maximum.

Required checks Between(2)(5)(2) and 5 -> true; 1 -> false; Between(5)(2) rejects.

Task 16 Separate delegate construction from invocation

Basic | Fix code | Currying

Fix Configure so the supplied test probe is not invoked until the returned function is called.

```
public static Func<int,int> Configure(Func<int,int> f)
{
    int sample=f(0); return x=>f(x)+sample;
}
```

Required checks Constructing function -> calls 0; invoke 3 -> expected/calls 1; invoke 4 -> calls 2.

Task 17 Complete generic composition

Basic | Complete TODO | Composition

Complete Compose<T, U, V> returning T -> V through first then second.

```
public static Func<T,V> Compose<T,U,V>(Func<T,U> first,Func<U,V> second)
{
    // TODO construct the composed function.
    throw new NotImplementedException();
}
```

Required checks int 3 -> double 6 -> string"6"; each stage invoked once on a call, zero times at construction.

Task 18 Two pricing policies in one workflow

Basic | Implement | Partial application

Implement PriceAll(Delivery[], Func<int, decimal>) producing id and cost values. Create standard 500+100*km and express 800+150*km.

Required checks Distance 2 -> 700 and 1100; distance 0 -> 500 and 800; original deliveries unchanged.

Task 19 Keep valid zero native results

Basic | Fix code | Native

Fix Convert so native zero is successful, not a rejection. The supplied library uses -1 for invalid input.

```
public static Result<int> Convert(int value)
    => value>0 ? new Result<int>.Ok(value)
               : new Result<int>.Fail("invalid");
```

Required checks Score(0,1) -> 0 -> Ok 0; Score(-1,1) -> -1 -> Fail.

Task 20 A partially applied percentage

Basic | Complete TODO | Partial application

Complete Discount(rate), requiring 0<=rate<=1, then receiving non-negative amount. Use decimal and no implicit rounding.

```
public static Func<decimal,decimal> Discount(decimal rate)
{
    // TODO validate rate and produce amount transformation.
    throw new NotImplementedException();
}
```

Required checks rate 0.1, amount 100 -> 90; rate 0 -> 100; invalid rate rejected.

Task 21 Curry and uncurry a function

Intermediate | Implement | Currying

Implement Curry(Func<int, int, int>) and Uncurry(Func<int, Func<int, int>>). Demonstrate agreement for addition and subtraction.

Required checks Round-trip subtract(10,3) -> 7; add(2,4) -> 6; preserve argument order.

Task 22 Do not share loop configuration

Intermediate | Fix code | Partial application

Fix the policy collection so each delegate retains its intended index value instead of the loop variable after completion.

```
public static Func<int,int>[] MakePolicies()
{
    var result=new Func<int,int>[3];
    for(int i=0;i<3;i++) result[i]=x=>x+i;
    return result;
}
```
Required checks Three policies applied to 10 -> 10,11,12 rather than all 13.

Task 23 A configurable scorer factory

Intermediate | Complete TODO | Currying

Complete Score as weight -> bonus -> Candidate -> int. Require weight 1..10 and bonus 0..100 before producing the final function.

```
public static Func<int,Func<int,Func<Candidate,int>>> Score()
{
    // TODO validate fixed parameters and return scorer.
    throw new NotImplementedException();
}
```
Required checks Weight 2, bonus 5, points 10 -> 25; invalid weight rejected; two configurations coexist.

Task 24 Compose selection scoring and ordering

Intermediate | Implement | Composition

Implement named pure stages for candidates: minimum points, supplied score, descending score then ordinal id. Do not mutate source.

Required checks A:40, B:40, C:20 with threshold 40 and score*2 -> A80, B80; no-match -> [].

Task 25 Match native parameter widths

Intermediate | Fix code | Native

Fix the binding so the C int32_t arguments and return are represented correctly in C#. Document why C long must not be assumed equivalent.

```
[System.Runtime.InteropServices.DllImport("practice",
    EntryPoint="weighted_score",
    CallingConvention=System.Runtime.InteropServices.CallingConvention.Cdecl)]
static extern long Score(long points,long weight);
```
Required checks 10,3 -> 30; 100,10 -> 1000 using the supplied int32_t contract.

Task 26 A safe native call adapter

Intermediate | Complete TODO | Native

Complete adapter: check documented ranges before calling native and convert -1 explicitly. Handle DllNotFoundException, EntryPointNotFoundException, and BadImageFormatException as distinct useful failures.

```
public static Result<int> Score(int points,int weight)
{
    // TODO validate, call ProvidedNative.Score, convert and catch relevant errors.
    throw new NotImplementedException();
}
```
Required checks 10,3 -> Ok 30; invalid input -> Fail/no call; controlled missing-library failure -> Fail.

Task 27 Pure extension for a supplied scoring function

Intermediate | Implement | Higher order

Implement an extension ScoreWith on IEnumerable<Candidate> accepting Func<Candidate, int>. Return a result snapshot in original order and leave source unchanged.

Required checks A10, B20 with*3 -> A30, B60; [] -> []; check both a lambda and named method.

Task 28 Do not enumerate at composition time

Intermediate | Fix code | Composition

Fix Prepare so constructing a workflow does not enumerate its source. Enumeration should happen only when invoking the returned workflow.

```
public static Func<int[]> Prepare(IEnumerable<int> source,Func<int,int> rule)
{
    var snapshot=source.Select(rule).ToArray();
    return ()=>snapshot;
}
```

Required checks Construction -> source visits 0; invoke -> visits equal source length; result matches supplied rule.

Task 29 Validate and capture a configuration copy

Intermediate | Complete TODO | Partial application

Complete CreatePrice from a mutable dictionary with keys base and rate. Validate values, copy them into stable local values, and return distance -> decimal.

```
public static Func<int,decimal> CreatePrice(Dictionary<string,decimal> config)
{
    // TODO validate and capture values, not the mutable dictionary.
    throw new NotImplementedException();
}
```

Required checks base 500/rate 100 gives 700 at 2; later dictionary rate 999 leaves 700; missing rate rejected.

Task 30 Curried multi-field validation

Intermediate | Implement | Currying

Implement allowedCategory -> maximumQuantity -> Product -> bool. Require positive maximum; product must match category prefix exactly, have positive quantity<=maximum and non-negative price.

Required checks Category A, max 5, A1/q 5/price 0 -> true; B1 -> false; quantity 6 -> false.

Task 31 Respect supplied transform order

Intermediate | Fix code | Composition

Fix Pipeline to accept an ordered list of int transformations and apply each once in supplied order. Empty list means identity.

```
public static int Pipeline(int value,Func<int,int>[] stages)
{
    foreach(var stage in stages.Reverse()) value=stage(value);
    return value;
}
```

Required checks Input 3,[+1,*2] -> 8; [] -> 3; each probe sees its predecessor output.

Task 32 Compose a curried policy with Result validation

Intermediate | Complete TODO | Composition

Complete Build: partially apply a price, validate non-negative distance into Result<int>, and map the successful distance through the configured price.

```
public static Result<decimal> Build(int km,Func<int,decimal> price)
{
    // TODO validate first, then apply only to success.
    throw new NotImplementedException();
}
```

Required checks Distance 2 -> Ok 700; -1 -> Fail/no pricing call; base 0/rate 0/km 0 -> Ok 0.

Task 33 Compare curried and factory interfaces

Intermediate | Implement | Currying

Implement both a curried price and an ordinary MakePrice factory producing identical distance-only functions. Document which calls supply configuration and which supply data.

Required checks Both standard configurations on distances 0,2,5 give 500,700,1000.

Task 34 Prevent duplicate native evaluation

Intermediate | Fix code | Native

Fix Compute so it invokes the native delegate once per request, then checks and uses the returned value.

```
public static Result<int> Compute(Func<int,int,int> native,int p,int w)
{
    if(native(p,w)<0) return new Result<int>.Fail("native");
    return new Result<int>.Ok(native(p,w)+10);
}
```
Required checks Valid request -> one call/correct total; rejection sentinel -> one call/failure.

Task 35 Document and implement native build lookup

Intermediate | Complete TODO | Native

Complete ResolvePath(runtimeFolder, platform). The folder must be absolute; platform is explicitly Windows, macOS or Linux. Choose practice.dll, libpractice.dylib or libpractice.so respectively, and return the combined absolute path. Reject unsupported platforms. Keep this path decision separate from building and loading.

```
public static string ResolvePath(string runtimeFolder, string platform)
{
    // TODO validate inputs and choose the filename from the explicit platform.
    throw new NotImplementedException();
}
```
Required checks Linux folder /tmp/n -> /tmp/n/libpractice.so; unsupported platform yields clear rejection.

Task 36 A higher-order fallback choice

Intermediate | Implement | Higher order

Implement Choose(primary, secondary, predicate) returning a Product -> decimal function. Select one policy per item; invoke only that policy.

Required checks Predicate true -> primary called 1/secondary 0; false -> reverse; document deterministic price results.

Task 37 Keep failed policy validation a failure

Intermediate | Fix code | Composition

Fix GetPolicy so a failed rate does not produce a free-price policy. Return Result<Func<int, decimal>> with meaningful failure.

```
public static Func<int,decimal> GetPolicy(decimal rate)
    => rate<0 ? _=>0 : km=>km*rate;
```
Required checks Rate -1 -> Fail; 0 -> Ok function returning 0; 2 -> Ok distance 3 giving 6.

Task 38 Curry a dependent predicate

Intermediate | Complete TODO | Currying

Complete MinimumStock -> MaximumPrice -> Product -> bool. Require non-negative configuration and use inclusive boundaries.

```
public static Func<int,Func<decimal,Func<Product,bool>>> Eligible()
{
    // TODO validate configuration and return the final predicate.
    throw new NotImplementedException();
}
```
Required checks stock 2, price 5 with min 2/max 5 -> true; stock 1 or price 6 -> false; invalid fixed settings rejected.

Task 39 Combine two pure policies

Intermediate | Implement | Composition

Implement CombineScores(Func<Candidate, int> a, Func<Candidate, int> b) returning their sum. Test supplied policies independently and together; preserve Candidate.

Required checks Points 10 with*2 and+5 -> 35; each invoked once; input remains 10.

Task 40 Bound partial application to valid configuration

Intermediate | Fix code | Partial application

Fix MakeDiscount to reject rate outside 0..1 before returning a function; never silently clamp it. Validate amounts separately when applying.

```
public static Func<decimal,decimal> MakeDiscount(decimal rate)
    => amount=>amount*(1-Math.Clamp(rate,0m,1m));
```

Required checks rate 1 -> amount 100 becomes 0; rate 1.2 and -0.1 rejected; amount -1 rejected.

Task 41 Build a configurable pipeline factory

Advanced | Complete TODO | Composition

Complete BuildPipeline(eligible, score, format) returning Candidate[] -> string[]. Select, score and order by score descending then id, and format. Delay all work until invocation.

```
public static Func<Candidate[],string[]> BuildPipeline(
    Func<Candidate,bool> eligible,Func<Candidate,int> score,
    Func<string,int,string> format)
{
    // TODO create the workflow without running it.
    throw new NotImplementedException();
}
```

Required checks A20, B10, C20 with minimum 20 -> A, C in tie order; no match -> []; no calls at construction.

Task 42 A native score inside a pure report boundary

Advanced | Implement | Native

Implement an adapter that computes native scores once, returns typed scored records, then passes them into a pure grouping report. Native errors must prevent report publication.

Required checks Candidates A10, B20 weight 3 -> scores 30,60, total 90; invalid candidate -> failure/no report.

Task 43 Fix a curried argument swap

Advanced | Fix code | Currying

Fix CurryPrice to interpret the first argument as base fee and second as rate. Validate both as non-negative and km as non-negative.

```
public static Func<decimal,Func<decimal,Func<int,decimal>>> CurryPrice
    => rate=>basis=>km=>basis+rate*km;
```

Required checks CurryPrice(500)(100)(2) -> 700; swapped interpretation would be 1100 and must fail the assertion.

Task 44 A configured optional lookup

Advanced | Complete TODO | Partial application

Complete MakeLookup(catalogue): validate unique ids, snapshot the source, return id -> Result<Product>. Later caller-list changes must not change the configured lookup.

```
public static Func<string,Result<Product>> MakeLookup(List<Product> catalogue)
{
    // TODO validate uniqueness and capture a copied index.
    throw new NotImplementedException();
}
```

Required checks Create with P1, clear caller list, lookup P1 still succeeds; P9 -> missing; duplicate ids rejected.

Task 45 Compare policy extension points

Advanced | Implement | Higher order

Implement equivalent scoring selection using a Func and a small strategy interface. Show equal results for two policies; explain how passing a function fits composition. Do not declare one approach universally faster.

Required checks Points 10 standard*2 -> 20; bonus*2+5 -> 25 under both approaches.

Task 46 Delivery pipeline with curried policies

Advanced | Integrate | Composition

Build selection, pricing and result stages using two partially applied curried policies. Include at least one custom extension, no input mutation, deterministic sorting, and assertions for negative distance rejection.

Required checks Distance 2 standard 700/express 1100; tied costs ordered by ordinal id; invalid distance produces explicit failure.

Task 47 Native candidate ranking

Advanced | Integrate | Native

Build and call weighted_score for valid candidates, translate failures, and rank by score descending then id. Include a real native call test and a controlled loading failure test.

Required checks A10, B20 weight 3 -> B60, A30; absent library -> clear failure; no managed fallback claimed as native execution.

Task 48 Configurable budget categorisation

Advanced | Integrate | Partial application

Create a validated snapshot of category limits, then return transaction -> Result<string> classification functions. Use two snapshots with different limits and a common report pipeline.

Required checks Amount 50 with limit 40 -> over; limit 60 -> within; changing original dictionary leaves configured results unchanged.

Task 49 A reusable pricing module

Advanced | Integrate | Currying

Create curried price and discount functions, partially apply them, and compose them with validated distance. Return a Result quote; include two configurations and explain each function type.

Required checks Standard distance 2 -> 700; applying 10% discount -> 630; invalid rate or distance -> failure before quote publication.

Task 50 Project policy refactor

Advanced | Integrate | Higher order

Replace a hard-coded or global-configured rule in your project with a useful higher-order API and two stable configured functions. Integrate one real native value if relevant, and document the boundary separately.

Required checks Record one identical input with two different expected outputs; add assertions proving caller data and configured behaviour remain stable.